Programare cu premeditare

Cătălin Frâncu

Prefață

Aici voi spune ceva introductiv.

- Problemele sînt ordonate după dificultate.
- Pentru Codeforces și Kilonova aveți nevoie de un cont pentru a vedea sursele.
- Am inclus legături către versiunile online ale surselor acolo unde evaluarea este publică. Rapoartele de evaluare arată și timpul și memoria.

Cuprins

I Grafuri **3**

1 Fluxuri **4**

1.1 Descrierea problemei 4

1.1.1 Exerciții teoretice 5

1.2 Metoda Ford-Fulkerson 5

1.2.1 Rețeaua reziduală 5

1.2.2 Drumuri de creștere 6

1.2.3 Metoda Ford-Fulkerson 6

1.3 Algoritmul Edmonds-Karp 6

1.3.1 Optimizare 7

1.4 Fluxuri și tăieturi 7

1.4.1 Aflarea tăieturii minime 9

1.5 Probleme 9

1.5.1 Problema Download Speed (CSES) 9

1.5.2 Problema Police Chase (CSES) 9

1.5.3 Problema Distinct Routes (CSES) 9

1.5.4 Problema Soldier and Traveling (Codeforces) 10

1.5.5 Problema Fox and Dinner (Codeforces) 10

1.5.6 Problema Asfalt (Lot 2011) 11

1.5.7 Problema Echipe (Lot 2025) 12

II Anexă: Cod-sursă **16**

A Grafuri - Fluxuri **17**

A.1 Problema Download Speed (CSES) 17

A.2 Problema Police Chase (CSES) 25

A.3 Problema Distinct Routes (CSES) 28

A.4 Problema Soldier and Traveling (Codeforces) 31

A.5 Problema Fox and Dinner (Codeforces) 35

A.6 Problema Asfalt (Lot 2011) 40

A.7 Problema Echipe (Lot 2025) 44

Partea I

Grafuri

Capitolul 1

Fluxuri

Voi încerca să nu încarc capitolele de fluxuri și cuplaje cu prea multă teorie, dar uneori nu putem scăpa. 🙂 Vom demonstra unele teoreme acolo unde ne ajută să înțelegem fenomenul.

Vom împrumuta imagini și probleme din monumentala carte Introduction to Algorithms de Cormen, Leiserson, Rivest și Stein, ediția a 4-a. Ca întotdeauna, vă încurajez să cumpărați o carte dacă vă este utilă, chiar dacă o puteți găsi și pe Internet sau pe torente. Vă recomand de asemenea să citiți și materialele de pe CP Algorithms, Flows and related problems și următoarele.

1.1 Descrierea problemei

Definiția 1 (Rețea de flux)**.** O rețea de flux este un graf orientat $G = (V, E)$ în care fiecare muchie (u, v) are o capacitate $c(u, v) \geq 0$. Există două noduri speciale în această rețea: o sursă s și o destinație t.

Exemple din viața reală:

- rețeaua de apă, de gaz sau de electricitate a unui oraș;
- o rețea de calculatoare care schimbă pachete între ele;
- o mină și o rețea de străzi care transportă minereul la fabrica de procesare.

În toate aceste exemple, capacitatea este un debit, respectiv cantitatea maximă de material / resurse / date care pot trece prin muchie într-o unitate de timp. De asemenea, în aceste rețele se aplică prima lege lui Kirchhoff: suma debitelor într-un nod este 0, cu excepția nodurilor s și t.

Definiția 2 (Flux)**.** Un flux de la s la t printr-o rețea este o funcție $f : E \to \mathbb{R}$ care satisface două proprietăți:

1. Respectarea capacităților: pentru orice muchie (u, v),

$$0 \leq f(u, v) \leq c(u, v)$$

2. Conservarea fluxului: pentru orice nod $u \in V - \{s, t\}$,

$$\sum_{v \in V} f(u, v) = \sum_{v \in V} f(v, u)$$

Definiția 3 (Valoarea fluxului)**.** Valoarea unui flux f este valoarea netă a fluxului care părăsește sursa s:

$$|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s)$$

Cu aceste definiții, problema fluxului maxim este: să se găsească un flux de valoare maximă într-o rețea de flux dată.

1.1.1 Exerciții teoretice

1. Cum tratăm cazul rețelelor cu surse și destinații multiple?
2. Cormen, exercițiul 24.1-6: dat fiind un graf $G = (V, E)$ și două noduri s și t, să se găsească două drumuri de la s la t astfel încît orice muchie $e \in E$ să apară pe cel mult un drum.
3. Cormen, exercițiul 24.1-7: cum tratăm cazul rețelelor care au capacități atît pe muchii, cît și în noduri? Exemplu din viața reală: routerele.

1.2 Metoda Ford-Fulkerson

1.2.1 Rețeaua reziduală

Neformal, rețeaua reziduală indusă de un flux f într-o rețea de flux ne arată, pe fiecare muchie, (1) cîte unități mai putem pompa (posibil 0) și (2) cîte unități mai putem scoate (posibil 0). Vom vedea exemple unde este util să reducem fluxul pe unele muchii pentru a-l putea mări pe altele, obținînd un nou flux de valoare mai mare.

Formal,

Definiția 4 (Rețea reziduală, capacități reziduale)**.** Rețeaua reziduală indusă de un flux f într-o rețea G se notează cu $G_f = (V, E_f)$ și are capacitățile reziduale c_f cu valorile:

$$c_f(u, v) = \begin{cases} c(u, v) - f(u, v) & \text{dacă } (u, v) \in E \\ f(v, u) & \text{dacă } (v, u) \in E \\ 0 & \text{altfel} \end{cases}$$

Remarcăm că muchiile reziduale (mulțimea E_f) se regăsesc printre muchiile originale sau inversele acestora.

Vom urmări figura 24.4 din Cormen pentru un exemplu.

1.2.2 Drumuri de creștere

Definiția 5 (Drum de creștere)**.** Dată fiind o rețea de flux G și un flux f în această rețea, un drum de creștere p este un flux în G_f.

Cu alte cuvinte, un drum de creștere **augmentează** fluxul existent, adică găsește un mod de a mai pompa niște unități de flux. Din nou, figura 24.4 din Cormen arată un drum de creștere. Cîte unități mai putem pompa? Răspunsul este minimul dintre capacitățile reziduale pe acel drum. Numim aceasta **capacitatea reziduală** a drumului de creștere și o notăm cu $c_f(p)$.

1.2.3 Metoda Ford-Fulkerson

Esența metodei este, pur și simplu: Cît timp există un drum de creștere p în rețeaua reziduală, augmentează fluxul cu $c_f(p)$ și recalculează rețeaua reziduală.

O denumim „metodă” și nu „algoritm” deoarece ea nu specifică detaliile găsirii drumului de creștere.

Vom citi o primă implementare, pentru problema Download Speed. Implementarea folosește DFS pentru a găsi drumuri de creștere. Cîteva detalii:

- Numărul de noduri în probleme de flux poate fi suficient de mic pentru a permite matrice de adiacență. Folosiți-le dacă preferați.
- Pentru o muchie (u, v) avem nevoie să găsim ușor muchia inversă (v, u), deoarece capacitatea reziduală pe o muchie scade cînd crește pe cealaltă. Eu am folosit faptul că muchiile ocupă celule adiacente în liste de adiacență. Dacă indicele uneia este pos, în mod garantat indicele celeilalte va fi `pos ^ 1`.
- Pe muchii stocăm doar capacitatea rămasă. Cum putem deduce din această informație fluxul efectiv pe fiecare muchie, fără a stoca informații suplimentare? Acest amănunt va deveni important la problemele de tăietură minimă.

Implementată astfel, metoda Ford-Fulkerson poate necesita $\mathcal{O}(|f|)$ iterații, fiecare în $\mathcal{O}(E)$, așa cum o arată figura 24.7 din Cormen. Mai mult, pentru valori iraționale (teoretice), este posibil ca algoritmul cu DFS să nu termine niciodată! De exemplu, la problema Maxflow de pe Infoarena testele 8, 9 și 10 durează foarte mult dacă le implementăm în $\mathcal{O}(n)$ per augmentare (pe testul 8 programul meu a durat 5 minute). Similar, testul 20 din problema Download Speed durează foarte mult.

1.3 Algoritmul Edmonds-Karp

Algoritmul Edmonds-Karp doar înlocuiește parcurgerea DFS cu una BFS, la fiecare căutare a unui drum de creștere. Prin aceasta, reușim să obținem o limită a numărului de iterații independentă de $|f|$, respectiv $\mathcal{O}(mn)$.

Definiția 6 (Muchie critică)**.** Muchia critică este muchia de capacitate minimă de pe un drum de creștere (cea care determină cantitatea pompată).

Schița demonstrației de complexitate a algoritmului Edmonds-Karp este:

1. Demonstrăm că distanțele minime $d(s, v)$ de la sursă la fiecare nod v nu scad niciodată pe măsură ce graful rezidual evoluează.
2. Esența demonstrației: Fie v nodul cel mai apropiat de s dintre toate cele ale căror distanțe au scăzut. Cum ar putea scădea distanța lui v de la o iterație f la următoarea iterație f'? Doar dacă în $G_{f'}$ a apărut o nouă muchie (u, v) care nu exista în G_f. Asta înseamnă că iterația f' a pompat flux în sensul $v \to u$. Dar algoritmul pompează flux doar în direcția BFS-ului. Așadar, v era strămoș al lui u în iterația f, dar a devenit succesor al lui u în iterația f', aceasta în condițiile în care $d(s, u)$ nu a scăzut. Așadar, distanța $d(s, v)$ de fapt a crescut, contradicție.
3. Demonstrăm că, pentru orice muchie (u, v), între două momente succesive cînd muchia devine critică, $d(s, u)$ crește cu cel puțin 2.
4. Distanțele nu pot fi niciodată mai mult de n.
5. Rezultă că o muchie nu poate fi critică de mai mult de $n/2$ ori, iar toate muchiile în total pot fi critice de $mn/2$ ori.

Așadar, algoritmul Edmonds-Karp va face cel mult $mn/2$ BFS-uri, avînd o complexitate totală de $\mathcal{O}(m^2n)$.

Implementarea este suficientă pentru problema Download Speed.

1.3.1 Optimizare

Infoarena menționează o optimizare care îmbunătățește timpii pentru problema Maxflow (dar nu și pentru problema Download Speed, unde chiar merge mai încet pe multe teste). Algoritmul calculează un arbore de părinți BFS cu efort $\mathcal{O}(m)$. Apoi încearcă să obțină beneficiul maxim din acest efort, căutînd mai multe drumuri de creștere. Nu ne oprim după ce găsim un drum de creștere prin predecesorul lui t în arborele BFS, ci căutăm drumuri de creștere pornind de la t prin **toți** predecesorii lui t. Unele dintre aceste căi vor avea valoarea 0, căci vor fi deja saturate. Totuși, algoritmul poate găsi mai multe drumuri de creștere.

Am inclus și o sursă cu această optimizare la problema Download Speed.

1.4 Fluxuri și tăieturi

Să considerăm acum problema Police Chase. Ea ne cere să eliminăm cît mai puține muchii dintr-un graf (neorientat, dar soluția este identică și în grafuri orientate) astfel încît să deconectăm un nod s de un nod t. Să vedem care este legătura acestei cerințe cu subiectul fluxurilor.

Am mai vorbit despre tăieturi în lecția despre arbori parțiali minimi. Vă reamintesc că o tăietură este o partiționare a grafului în două mulțimi de noduri disjuncte. Cînd vorbim despre fluxuri, ne interesează acele tăieturi care separă sursa s de destinația t. Convențional, notăm cele două mulțimi cu S și T.

Definiția 7 (Fluxul net)**.** Fluxul net printr-o tăietură (S, T) este

$$\sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u)$$

Teorema 1. Pentru un flux dat f, valoarea fluxului net prin orice tăietură $S - T$ este $|f|$.

O demonstrație facilă este inductivă. Pentru $S = \{s\}$ și $T = V - \{s\}$, în mod evident fluxul este $|f|$. Acum, să presupunem că printr-o tăietură $S - T$ avem fluxul net $|f|$. Să „tragem" de frontieră pentru a muta exact un nod u din T în S, deci fie $S' = S \cup \{u\}$ și $T' = T - \{u\}$. Ce s-a modificat? Vom vedea că am pierdut și am cîștigat exact muchii care intră și ies din u, iar suma contribuțiilor lor (datorită conservării fluxului) este 0. Deci valoarea fluxului net prin tăieturile $S - T$ și $S' - T'$ este aceeași.

Definiția 8 (Capacitatea unei tăieturi)**.** Capacitatea unei tăieturi (S, T) este

$$\sum_{u \in S} \sum_{v \in T} c(u, v)$$

Este relativ clar că prin nicio tăietură nu putem transmite mai mult flux decît capacitatea tăieturii (pe unde s-ar duce?). Așadar, fluxul maxim este mai mic sau egal cu capacitatea oricărei tăieturi. Alternativ, fluxul maxim este mai mic sau egal cu capacitatea tăieturii minime. Următoarea teoremă arată că, de fapt, cele două valori sînt chiar egale.

Teorema 2 (Max-flow min-cut)**.** Următoarele trei condiții sînt echivalente:

1. f este un flux maxim în G.
2. Nu există niciun drum de creștere în rețeaua reziduală G_f.
3. Există o tăietură (S, T) în G pentru care $c(S, T) = |f|$.

Demonstrația este circulară. Le enunțăm pe cele simple primele:

(1) $\implies$ (2): Dacă ar exista un drum de creștere, l-am putea folosi ca să creștem fluxul, deci f nu ar fi maxim.

(3) $\implies$ (1): Știm deja că orice flux este cel mult egal cu orice tăietură. Deci, dacă găsim un flux egal cu o tăietură, atunci fluxul este maxim.

(2) $\implies$ (3): Esența demonstrației este alegerea tăieturii. Nu întîmplător, fie S mulțimea nodurilor care încă sînt accesibile din s în G_f și fie T mulțimea nodurilor inaccesibile. Desigur, $t \in T$ căci altfel t ar fi accesibil din S și ar contrazice (2). Fie acum orice pereche de noduri $u \in S$ și $v \in T$. Iau naștere trei cazuri posibile:

1. Dacă $(u, v) \in E$, atunci obligatoriu muchia este saturată, altfel am mai putea pompa flux pe ea, deci $(u, v) \in G_f$, deci $v \in S$, contradicție. Așadar, $f(u, v) = c(u, v)$.
2. Dacă $(v, u) \in E$, atunci obligatoriu muchia este nefolosită, altfel am mai putea scoate flux pe ea, deci $c_f(u, v) > 0$, deci $v \in S$, contradicție. Așadar, $f(u, v) = 0$.

3. Altfel nu există muchii între u și v și automat $f(u,v) = f(v,u) = 0$.

Atunci fluxul net prin tăietura (S,T) este:

$$|f| = f(S,T) = \sum_{u \in S} \sum_{v \in T} f(u,v) - \sum_{u \in S} \sum_{v \in T} f(v,u) = \sum_{u \in S} \sum_{v \in T} c(u,v) - \sum_{u \in S} \sum_{v \in T} 0 = c(S,T)$$

1.4.1 Aflarea tăieturii minime

Implementarea (care rezolvă problema Mincut pe NerdArena) cere doar 1% efort suplimentar după găsirea fluxului maxim. Ultimul BFS, cel care constată că destinația t nu mai este accesibilă, lasă în cîmpul parent valori pozitive pentru nodurile accesibile, dar valoarea specială NONE pentru noduri inaccesibile. Trebuie doar să tipărim muchiile care unesc noduri accesibile cu noduri inaccesibile.

Există totuși un detaliu semnificativ. Să considerăm figura 24.6 din Cormen. Fluxul maxim este 23. În ultima rețea reziduală, tăietura este $S = \{s, v_1, v_2, v_4\}$ și $T = \{v_3, t\}$. Dar suma muchiilor care traversează tăietura este $12+9+7+4 = 32$. De unde apare discrepanța? Este important să ne amintim că pe muchia v_3, v_2 nu circulă flux de valoare 9, ci avem o capacitate de 9, nefolosită! Prin contrast, pe muchiile $12+7+4$ chiar circulă flux. Cu alte cuvinte, este vital să adunăm doar fluxul efectiv (muchii originale saturate), nu capacitățile reziduale pe care le stocăm de regulă în noduri.

1.5 Probleme

1.5.1 Problema Download Speed (CSES)

enunț • surse

Problema este educațională și necesită aflarea fluxului maxim.

1.5.2 Problema Police Chase (CSES)

enunț • sursă

Problema este educațională și necesită aflarea tăieturii minime. Străzile au capacitate 1 și sînt bidirecționale. Pentru implementare, este suficient să creăm două muchii de capacitate 1, astfel încît fluxul să poată fi 1 în orice sens (și 1 rezidual în sensul opus). Nu este nevoie să creăm două perechi de muchii.

O problemă 99% identică este Mincut (NerdArena). Diferența minoră este că muchiile sînt unidirecționale și pot avea capacități naturale. Iată o sursă online.

1.5.3 Problema Distinct Routes (CSES)

enunț • sursă

Ideea nu este grea. Privim fiecare teleportor ca pe o muchie de capacitate 1. În acest fel, fiecare unitate de flux pe care o putem pompa va merge pe o cale diferită. La final, colectăm și eliminăm rutele una cîte una.

Întrebare: Am putea folosi algoritmul Ford Fulkerson naiv, cu DFS? Avantajul ar fi că putem colecta rutele imediat ce le găsim. Nu este nicio problemă, căci oricum ieșirea problemei este proporțională cu mărimea fluxului. Nu-i așa?

1.5.4 Problema Soldier and Traveling (Codeforces)

enunț • sursă

De regulă problemele de flux nu sînt formulate ca atare, ci trebuie să le reduceți la calculul unui flux. De asemenea, un mecanism întîlnit frecvent este dublarea nodurilor. Pentru problema de față,

- Construim nodurile $x_1, x_2, \ldots, x_n$ și $y_1, y_2, \ldots, y_n$, așezate vizual pe două coloane (x-ii în stînga, y-ii în dreapta). Conceptual, x este configurația trupelor înainte de mutare, iar y după mutare.
- Modelăm muchiile grafului original: pentru fiecare muchie (u, v) trasăm muchiile (x_u, y_v) și (x_v, y_u), de capacitate infinită. Astfel arătăm că oricîți soldați se pot muta între orașe pe muchiile existente.
- Trasăm muchii între fiecare pereche (x_i, y_i), de asemenea de capacitate infinită. Astfel arătăm că oricîți soldați pot rămîne pe loc în orașul i.
- Adăugăm o sursă s și o unim cu fiecare x_i printr-o muchie de capacitate a_i. Astfel arătăm că numărul total de soldați din orașul i este a_i și pentru fiecare dintre ei trebuie să decidem o acțiune. Acțiunea se traduce prin pomparea unei unități de flux, fie în y_i fie într-un oraș adiacent.
- Similar, adăugăm o destinație t și unim fiecare y_i cu aceasta printr-o muchie de capacitate b_i.

Cu această reprezentare, problema are soluție dacă și numai dacă există un flux de capacitate $\sum a_i$ și în plus $\sum a_i = \sum b_i$. Descrierea traseelor unităților de flux nu este unică. Evaluatorul acceptă orice soluție.

1.5.5 Problema Fox and Dinner (Codeforces)

enunț • sursă

Cum modelăm această problemă ca pe una de flux?

Să observăm că orice ciclu va alterna între numere pare și numere impare. Altfel, dacă alăturăm două numere pare sau două numere impare, suma va fi pară, deci nu va putea fi primă. Așadar, o primă condiție este ca n să fie par, iar numerele să fie jumătate pare și jumătate impare.

Ne vine astfel ideea să trasăm muchii între fiecare pereche de numere de sumă primă. Aceasta

împarte natural graful în două mulțimi sau coloane: numerele pare la stînga, numerele impare la dreapta. Acum orice muchie va fi orientată de la stînga la dreapta. Să acordăm tuturor muchiilor capacitate 1.

Acum, dorim să producem cicluri de lungime cel puțin 4, căci enunțul interzice ciclurile de lugime 2. Rezultă că fiecare nod trebuie conectat cu exact două noduri din tabăra opusă (vecinii săi de la masă). De aceea, să adăugăm o sursă pe care o unim prin muchii de capacitate 2 cu toate numerele pare. Similar, să adăugăm o destinație pe care o unim prin muchii de capacitate 2 cu toate numerele impare.

Rețeaua este acum gata și trebuie să vedem dacă putem pompa n unități de flux prin ea.

Algoritmul Edmonds-Karp simplu (fără încercarea de optimizare) pare să fie suficient. Este posibil ca și metoda Ford-Fulkerson să fie suficientă, căci fluxul maxim este 100.

La final, trebuie să colectăm ciclurile rezultate. Remarcăm că de la stînga la dreapta mergem pe muchii saturate (capacitate reziduală 0), iar de la dreapta la stînga revenim pe muchii reziduale (capacitate reziduală 1). Astfel, putem să iterăm prin toate valorile pare și, pentru fiecare valoare încă neexplorată, să urmăm muchiile pînă închidem un ciclu. Deoarece pentru orice unitate de flux la intrare există și una la ieșire, acest algoritm nu se va bloca niciodată, ci va găsi toate ciclurile.

1.5.6 Problema Asfalt (Lot 2011)

enunț • sursă

După formalizare, înțelegem că problema ne cere să identificăm un set minim de muchii care să acopere toate drumurile minime de la S la D. Problema mi-a amintit de problemele President and Roads și Edges in MST, care cereau clasificarea muchiilor în drumuri minime, respectiv în arborii parțiali minimi.

Dacă problema încă pare greu de „apucat”, să ne gîndim la cazuri particulare. Dacă există o punte între S și D, atunci în mod necesar ea este parte din toate drumurile minime, deci setul constă doar din ea. Dar nu este atît de simplu. Poate exista o muchie e care nu este factual punte (face parte dintr-un ciclu), dar restul muchiilor de pe ciclu sînt atît de scumpe, încît practic toate drumurile minime trec prin e. Și atunci soluția constă doar din e.

Așa ajungem la ideea de clasificare a muchiilor în trei categorii (în practică, doar două):

1. Muchii care nu fac parte din niciun drum minim. Acestea nu vor face parte niciodată din soluție, căci scumpirea lor nu afectează drumurile minime.
2. Muchiile care fac parte din orice drum minim. Oricare dintre acestea poate constitui, de una singură, o soluție.
3. Muchiile care fac parte din unele drumuri minime. Acestea trebuie combinate cu alte muchii pentru a găsi o soluție de cardinalitate cît mai mică.

De aici ne vine ideea tăieturii. Aruncăm muchiile din categoria 1 și le păstrăm doar pe cele din

categoriile 2 și 3. Vă mai amintiți cum le depistăm? Facem două măsurători de distanțe cu algoritmul Dijkstra, odată pornind din S și o dată din D. Apoi, o muchie (u, v) este pe cel puțin un drum minim dacă și numai dacă

$$d(S, u) + c(u, v) + d(v, D) = d(S, D)$$

Pe acest graf nu ne mai interesează distanțele. Ne interesează să găsim un număr minim de muchii care să deconecteze S de D. Este exact problema tăieturii minime.

Un detaliu de implementare: Am ales să refolosesc muchiile grafului inițial. Cîmpul `c`, inițial folosit pentru distanțe în algoritmul lui Dijkstra, stochează capacități în algoritmul Edmonds-Karp. Muchiile care fac parte din rețeaua de flux capătă capacitate 1 doar în sensul indicat de egalitatea de mai sus. Muchiile care nu fac parte din rețeaua de flux capătă capacitate 0 în ambele sensuri.

După cum am vorbit anterior, tăietura minimă constă doar din fluxul pe muchiile directe, nu și pe cele reziduale. Spre deosebire de problema Mincut, unde încă de la citirea grafului știam că muchiile pare sînt directe, iar cele impare sînt reziduale (chiar noi le atribuiam astfel), aici ele pot surveni oricum, după cum se nimeresc distanțele. Așadar, în rutina `convert_to_flow_network()` am decis să setez explicit un cîmp boolean forward pe muchiile directe.

1.5.7 Problema Echipe (Lot 2025)

enunț • sursă

Iată o problemă extrem de dură. În concurs, doar Mihai Voicu a obținut 100 de puncte. Următoarele punctaje au fost 45, 23, 14 și restul sub 9. Deci nu ne punem problema să o înțelegem și să o codăm repede, ci doar să găsim o soluție în ritmul nostru (a mea are 300 de linii efective și mi-a consumat cam două zile).

Găsirea numărului de echipe

Compatibilitatea oamenilor în echipe ține doar de atributele lor, nu de risc. De aceea, numărul maxim de echipe, să-i spunem E, este constant și îl putem calcula o singură dată. La schimbarea scorului, reorganizăm echipele dar nu adăugăm și nu ștergem echipe.

Sper că de acum începeți să intuiți reducerea la flux: mapăm oamenii la rolurile pe care le pot îndeplini. Deoarece ar fi mult prea scump să construim un graf cu $\mathcal{O}(n)$ noduri, vom colecta doar frecvențele diferitelor categorii (combinații de atribute), numerotate de la 1 la 7 conform scrierii binare a atributelor. Apoi construim un graf cu:

- O mulțime A cu 7 noduri corespunzătoare celor 7 categorii.
- O mulțime B cu 4 noduri corespunzătoare celor 4 roluri (C, I, M, I/M).
- Un nod sursă s și un nod destinație t.
- Muchii de capacitate ∞ (sau n) de la fiecare categorie din A la rolurile compatibile din B.

- Muchii de la s la A, capacitatea fiecărei muchii fiind egală cu numărul de oameni din acea categorie.
- Pentru început, muchii de capacitate 0 de la B la t.

Pe acest graf dorim să găsim un flux maxim, adică să cuplăm cît mai multe persoane cu cele 4 roluri. Dar atenție! Fluxul pe cele 4 muchii de la B la t trebuie să fie egal, deoarece trebuie să avem la fel de multe persoane în fiecare rol. O implementare normală de flux nu garantează acest lucru. De aceea, procedăm astfel, în mod repetat:

- Crește cu cîte 1 capacitatea pe toate muchiile de la B la t.
- Refă fluxul și vezi dacă el a crescut cu 4.

Ne oprim cînd nu mai putem crește fluxul cu 4.

Care este complexitatea acestui prim pas? Deoarece graful are 13 noduri, putem considera parcurgerile acestui graf ca $\mathcal{O}(1)$. Folosim cea mai simplă metodă (Ford-Fulkerson cu DFS), care va face cel mult $n/4$ iterații. Complexitatea este $\mathcal{O}(n)$.

Ca implementare, am dat nume celor 13 noduri și am hardcodat matricea de adiacență. Am încapsulat tot codul referitor la flux într-o clasă `network_flow`, care ocupă circa 700 de octeți, suficient de mică încît să o putem copia în întregime. Restul programului folosește fluxul ca primitivă (operație de bază).

Găsirea riscului minim pentru riscurile inițiale

Aici orice abordare bazată pe flux este sortită eșecului. Toate riscurile pot diferi, deci nu mai putem grupa nodurile, iar algoritmii practici de flux cedează mult înainte de $n = 150\,000$.

Să ne reamintim, în schimb, că ne permitem să facem $\mathcal{O}(n)$ fluxuri în graful de 13 noduri. Atunci putem încerca următoarea abordare greedy. Considerăm oamenii în ordine descrescătoare a riscului. Cînd considerăm persoana X din categoria A_X, încercăm să renunțăm la ea. Cu alte cuvinte, ne întrebăm: Dacă X n-ar exista, am mai putea găsi un flux cu E echipe? Dacă da, renunțăm la X. Altfel îl includem pe X în echipele inițiale.

Iată o schiță de demonstrație de corectitudine, prin reducere la absurd. Fie soluția optimă S, care îl include pe X. Să presupunem că algoritmul greșește, că renunță la X și obține o soluție neoptimă S' de risc mai mare decît S. În S' există o persoană X' care îndeplinește rolul lui X. Atunci putem să pornim de la S și să-l schimbăm pe X cu X' pentru a obține o configurație de risc și mai mic. Acest lucru este imposibil întrucît am presupus că X este optim.[1]

Acum să vedem cum implementăm algoritmul. Nu dorim să recalculăm fluxul de la 0 pentru fiecare dintre cei n oameni, pentru că am obține complexitatea $\mathcal{O}(n^2)$. Trebuie să adaptăm în permanență fluxul existent. Iau naștere două cazuri:

[1] Am spus „schiță” deoarece, de fapt, este posibil ca X' să fie deja în S, dacă diferența între S și S' presupune o reorganizare mai mare. În acel caz trebuie să căutăm persoana X'' care îl înlocuiește pe X' în S'. Astfel poate lua naștere un lanț de persoane, dar invariabil vom ajunge la o persoană care este disponibilă.

1. Dacă fluxul existent **nu** saturează muchia (s, A_X), atunci îl putem pur și simplu sări pe X deoarece există suficiente alte persoane din categoria A_X cu care să formăm echipe. De asemenea, reducem capacitatea muchiei cu 1. Astfel reflectăm faptul că nu putem sări oricîte persoane din categoria A_X, ci la un moment dat muchia va deveni saturată.
2. Dacă fluxul existent saturează muchia (s, A_X), atunci identificăm traseul unei unități de flux $s \to A_x \to B_y \to t$ și o eliminăm din graf. Nu creștem capacitatea pe muchia (s, A_x), deoarece dorim ca fluxul să ne dea o altă persoană, nu tot pe X. Apoi încercăm să pompăm o unitate de flux. Dacă putem, înseamnă că noul flux a reușit să reorganizeze persoanele pentru a obține $4E$ echipe, iar pe X îl putem sări.

Actualizările

Să ne gîndim la cele 4 scenarii care pot apărea cînd riscul lui X se modifică.

1. Dacă X era parte din echipe, iar riscul îi scade, atunci el va rămîne în echipe. Ar fi absurd să îl înlocuim acum – cu atît mai mult am fi avut motiv să îl înlocuim înainte de scăderea riscului.
2. Dacă X nu era parte din echipe, iar riscul îi crește, atunci el va rămîne în afara echipelor.
3. Dacă X era parte din echipe, iar riscul îi crește, atunci ar putea fi profitabil să îl înlocuim.
4. Dacă X nu era parte din echipe, iar riscul îi scade, atunci ar putea fi profitabil să îl aducem în echipe în locul altcuiva.

Putem demonstra că în scenariile (3) și (4) este necesară cel mult o înlocuire. Esența raționamentului este următoarea: Dacă prin modificarea riscului lui X devin necesare $k > 1$ înlocuiri pentru a reface soluția optimă, atunci puteam face $k - 1$ dintre aceste înlocuiri și înainte ca riscul lui X să se modifice și am fi obținut o soluție mai bună la acel moment.

Dar cum implementăm asta? Sînt doi pași, unul care ia în calcul doar categoria lui X, A_X, și altul care ia în calcul toate categoriile.

Așadar, la pasul 1 analizăm cîte persoane din categoria A_X are soluția (concret, care este fluxul pe muchia (s, A_X). Fie k acest număr. Desigur că în soluție noi vom folosi persoanele cu cele mai mici k riscuri din A_x. Să menținem această informație în două multiseturi: riscurile persoanelor din soluție și riscurile persoanelor din afara soluției. (Clarificare: stocăm cîte două multiseturi pentru fiecare dintre cele 7 categorii).

Acum, cînd riscul lui X se schimbă, îl căutăm în multisetul său și îi actualizăm valoarea. Dacă în urma acestei actualizări multiseturile se încalecă, adică cea mai riscantă persoană din soluție depășește cea mai puțin riscantă persoană din afara soluției, le interschimbăm și recalculăm riscul total.

La pasul 2 ținem cont de reorganizări mai mari care pot deveni necesare, afectînd distribuția pe categorii. De exemplu, cînd riscul lui X crește, ideal am vrea să eliminăm cea mai scumpă persoană din clasa A_X și să-i spunem rețelei de flux „refă fluxul”. Din păcate, rețeaua de flux habar nu are despre riscuri. Este foarte posibil să ne dea înapoi fix persoana din clasa A_X sau o

altă persoană de risc mare.

În loc de aceasta, vom încerca rînd pe rînd fiecare altă categorie A_Y. Concret,

- Dacă riscul lui X a scăzut, vom încerca pe rînd să ștergem o persoană din fiecare (altă) categorie A_Y și să inserăm o persoană din categoria A_X.
- Dacă riscul lui X a crescut, vom încerca pe rînd să inserăm o persoană din fiecare (altă) categorie A_Y și să ștergem o persoană din categoria A_X.

Pentru a ghida fluxul să ne dea categoria dorită, vom descrește cu 1 capacitatea categoriei eliminate și vom crește cu 1 capacitatea categoriei adăugate. Iată un mic fragment din cod, cel care încearcă să înlocuiască o persoană din categoria `del` cu o persoană din categoria `ins`.

```
// Încearcă să înlocuiască ultimul element inclus din del cu primul element
// exclus din ins. Operează pe graful gc, pe care îl inițializează cu o copie
// a lui g.
//
// Returnează noul risc, sau INF dacă înlocuirea este imposibilă sau neoptimă.
long long try_swap_pair(int del, int ins, network_flow* gc, long long best_so_far) {
  // Categoriile trebuie să fie diferite și trebuie să existe persoane de
  // șters și de adăugat.
  if (del == ins ||
      tc.risks_in[del].empty() ||
      tc.risks_out[ins].empty()) {
    return INF;
  }

  // Noul risc pe care l-am obține prin schimbare trebuie să fie o optimizare.
  int del_risk = *tc.risks_in[del].rbegin();
  int ins_risk = *tc.risks_out[ins].begin();
  long long new_risk = tc.total_in_risk - del_risk + ins_risk;

  if (new_risk >= best_so_far) {
    return INF; // Este o schimbare în rău.
  }

  // Acum are sens să încercăm să refacem fluxul.
  *gc = g;
  gc->remove_unit(del);
  gc->increase_source_capacity(ins);
  gc->ford_fulkerson();

  return (gc->flow == 4 * num_teams) ? new_risk : INF;
}
```

Complexitatea actualizărilor este $O(q \log n)$ datorită operațiilor pe multiseturi.

Partea II

Anexă: Cod-sursă

Anexa A

Grafuri - Fluxuri

A.1 Problema Download Speed (CSES)

‹ înapoi

Sursă cu metoda Ford-Fulkerson (DFS).

```
// Metodă: Ford Fulkerson + DFS.
#include <stdio.h>

const int MAX_NODES = 1'000;
const int MAX_EDGES = 5'000;
const int NIL = -1;
const int INFINITY = 1'000'000'001;

struct edge {
  int cap;
  short v, next;
};

struct node {
  short adj;
  bool seen;
};

edge e[2 * MAX_EDGES];
node nd[MAX_NODES + 1];
int n;

void add_edge(short u, short v, int cap) {
  static int pos = 0;

  e[pos] = { cap, v, nd[u].adj };
  nd[u].adj = pos++;
}
```

```
void read_data() {
  int m;
  scanf("%d %d", &n, &m);
  for (int u = 1; u <= n; u++) {
    nd[u].adj = NIL;
  }
  while (m--) {
    int u, v, c;
    scanf("%d %d %d", &u, &v, &c);
    add_edge(u, v, c);
    add_edge(v, u, 0);
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

void reset_dfs() {
  for (int u = 1; u <= n; u++) {
    nd[u].seen = false;
  }
}

// Găsește un drum de creștere de la u la destinație și pompează cel mult
// max_pump unități pe el. Returnează fluxul pompat efectiv sau 0 dacă nu
// există niciun drum de creștere.
int dfs(int u, int max_pump) {
  nd[u].seen = true;
  if (u == n) {
    return max_pump;
  }

  int pumped = 0;
  for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
    int v = e[pos].v;
    int residual = e[pos].cap;
    if (!pumped && residual && !nd[v].seen) {
      pumped = dfs(v, min(max_pump, residual));
      if (pumped) {
        e[pos].cap -= pumped;
        e[pos ^ 1].cap += pumped;
      }
    }
  }

  return pumped;
}
```

```
long long ford_fulkerson() {
  long long flow = 0, augment;

  do {
    reset_dfs();
    augment = dfs(1, INFINITY);
    flow += augment;
  } while (augment);

  return flow;
}

int main() {
  read_data();
  long long max_flow = ford_fulkerson();
  printf("%lld\n", max_flow);

  return 0;
}
```

Sursă cu algoritmul Edmonds-Karp.

```
// Algoritmul Edmonds-Karp (Ford-Fulkerson + BFS).
#include <stdio.h>

const int MAX_NODES = 1'000;
const int MAX_EDGES = 5'000;
const int NIL = -1;
const int INFINITY = 1'000'000'001;

struct edge {
  int cap;
  short v, next;
};

struct node {
  short adj;
  short parent, edge;
};

struct queue {
  int v[MAX_NODES];
  int head, tail;

  void clear() {
    head = tail = 0;
  }

  void enqueue(int x) {
```

```
    v[tail++] = x;
  }

  int dequeue() {
    return v[head++];
  }

  bool is_empty() {
    return head == tail;
  }
};

edge e[2 * MAX_EDGES];
node nd[MAX_NODES + 1];
queue q;
int n;

void add_edge(short u, short v, int cap) {
  static int pos = 0;

  e[pos] = { cap, v, nd[u].adj };
  nd[u].adj = pos++;
}

void read_data() {
  int m;
  scanf("%d %d", &n, &m);
  for (int u = 1; u <= n; u++) {
    nd[u].adj = NIL;
  }
  while (m--) {
    int u, v, c;
    scanf("%d %d %d", &u, &v, &c);
    add_edge(u, v, c);
    add_edge(v, u, 0);
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

bool bfs_reachable() {
  for (int u = 1; u <= n; u++) {
    nd[u].parent = NIL;
  }
  q.clear();
  q.enqueue(1);

  while (!q.is_empty() && (nd[n].parent == NIL)) {
```

```
    int u = q.dequeue();
    for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
      int v = e[pos].v;
      // Consideră doar capacitățile pozitive către noduri neexplorate.
      if ((nd[v].parent == NIL) && e[pos].cap) {
        nd[v].parent = u;
        nd[v].edge = pos;
        q.enqueue(v);
      }
    }
  }

  return (nd[n].parent != NIL);
}

int path_minimum() {
  int min_cap = INFINITY;
  int u = n;
  while (u != 1) {
    min_cap = min(min_cap, e[nd[u].edge].cap);
    u = nd[u].parent;
  }
  return min_cap;
}

void pump_on_path(int flow) {
  int u = n;
  while (u != 1) {
    e[nd[u].edge].cap -= flow;
    e[nd[u].edge ^ 1].cap += flow;
    u = nd[u].parent;
  }
}

long long ford_fulkerson() {
  long long flow = 0;

  while (bfs_reachable()) {
    int augment = path_minimum();
    pump_on_path(augment);
    flow += augment;
  }

  return flow;
}

int main() {
  read_data();
  long long max_flow = ford_fulkerson();
  printf("%lld\n", max_flow);
```

```
  return 0;
}
```

Sursă cu algoritmul Edmonds-Karp cu drumuri de creștere multiple.

```
// Algoritmul Edmonds-Karp cu o încercare de optimizare. La fiecare iterație
// încercăm să găsim mai multe drumuri de creștere din același arbore BFS.
#include <stdio.h>

const int MAX_NODES = 1'000;
const int MAX_EDGES = 5'000;
const int NIL = -1;
const int INFINITY = 1'000'000'001;

struct edge {
  int cap;
  short v, next;
};

struct node {
  short adj;
  short parent, edge;
};

struct queue {
  int v[MAX_NODES];
  int head, tail;

  void clear() {
    head = tail = 0;
  }

  void enqueue(int x) {
    v[tail++] = x;
  }

  int dequeue() {
    return v[head++];
  }

  bool is_empty() {
    return head == tail;
  }
};

edge e[2 * MAX_EDGES];
node nd[MAX_NODES + 1];
queue q;
```

```
int n;

void add_edge(short u, short v, int cap) {
  static int pos = 0;

  e[pos] = { cap, v, nd[u].adj };
  nd[u].adj = pos++;
}

void read_data() {
  int m;
  scanf("%d %d", &n, &m);
  for (int u = 1; u <= n; u++) {
    nd[u].adj = NIL;
  }
  while (m--) {
    int u, v, c;
    scanf("%d %d %d", &u, &v, &c);
    add_edge(u, v, c);
    add_edge(v, u, 0);
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

bool bfs_reachable() {
  for (int u = 1; u <= n; u++) {
    nd[u].parent = NIL;
  }
  q.clear();
  q.enqueue(1);

  while (!q.is_empty() && (nd[n].parent == NIL)) {
    int u = q.dequeue();
    for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
      int v = e[pos].v;
      // Consideră doar capacitățile pozitive către noduri neexplorate.
      if ((nd[v].parent == NIL) && e[pos].cap) {
        nd[v].parent = u;
        nd[v].edge = pos;
        q.enqueue(v);
      }
    }
  }

  return (nd[n].parent != NIL);
}
```

```
int path_minimum() {
  int min_cap = INFINITY;
  int u = n;
  while (u != 1) {
    min_cap = min(min_cap, e[nd[u].edge].cap);
    u = nd[u].parent;
  }
  return min_cap;
}

void pump_on_path(int flow) {
  int u = n;
  while (u != 1) {
    e[nd[u].edge].cap -= flow;
    e[nd[u].edge ^ 1].cap += flow;
    u = nd[u].parent;
  }
}

long long ford_fulkerson() {
  long long flow = 0;

  while (bfs_reachable()) {
    // Succesorii destinației sînt și predecesorii ei.
    for (int pos = nd[n].adj; pos != NIL; pos = e[pos].next) {
      int v = e[pos].v;
      if ((v == 1) || (nd[v].parent != NIL)) {
        // Conectează destinația la v în arborele BFS.
        nd[n].parent = v;
        nd[n].edge = pos ^ 1;
        int augment = path_minimum();
        if (augment) {
          pump_on_path(augment);
          flow += augment;
        }
      }
    }
  }

  return flow;
}

int main() {
  read_data();
  long long max_flow = ford_fulkerson();
  printf("%lld\n", max_flow);

  return 0;
}
```

A.2 Problema Police Chase (CSES)

‹ înapoi

Sursă cu algoritmul Edmonds-Karp.

```
#include <stdio.h>

const int MAX_NODES = 500;
const int MAX_EDGES = 1'000;
const int NIL = -1;
const int INFINITY = 1'000'000;

struct edge {
  short v;
  short cap;
  short next;
};

struct node {
  short adj;
  short parent, edge;
};

struct queue {
  int v[MAX_NODES];
  int head, tail;

  void clear() {
    head = tail = 0;
  }

  void enqueue(int x) {
    v[tail++] = x;
  }

  int dequeue() {
    return v[head++];
  }

  bool is_empty() {
    return head == tail;
  }
};

edge e[4 * MAX_EDGES];
node nd[MAX_NODES + 1];
queue q;
int n;
```

```
void add_edge(short u, short v) {
  static int pos = 0;

  e[pos] = { v, 1, nd[u].adj };
  nd[u].adj = pos++;
}

void read_data() {
  int m;
  scanf("%d %d", &n, &m);
  for (int u = 1; u <= n; u++) {
    nd[u].adj = NIL;
  }
  while (m--) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_edge(u, v);
    add_edge(v, u);
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

bool bfs_reachable() {
  for (int u = 1; u <= n; u++) {
    nd[u].parent = NIL;
  }
  q.clear();
  q.enqueue(1);

  while (!q.is_empty() && (nd[n].parent == NIL)) {
    int u = q.dequeue();
    for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
      int v = e[pos].v;
      // Consideră doar capacitățile pozitive către noduri neexplorate.
      if ((nd[v].parent == NIL) && e[pos].cap) {
        nd[v].parent = u;
        nd[v].edge = pos;
        q.enqueue(v);
      }
    }
  }

  return (nd[n].parent != NIL);
}

int path_minimum() {
  int min_cap = INFINITY;
```

```
  int u = n;
  while (u != 1) {
    min_cap = min(min_cap, e[nd[u].edge].cap);
    u = nd[u].parent;
  }
  return min_cap;
}

void pump_on_path(int flow) {
  int u = n;
  while (u != 1) {
    e[nd[u].edge].cap -= flow;
    e[nd[u].edge ^ 1].cap += flow;
    u = nd[u].parent;
  }
}

int ford_fulkerson() {
  int flow = 0;

  while (bfs_reachable()) {
    int augment = path_minimum();
    pump_on_path(augment);
    flow += augment;
  }

  return flow;
}

void write_min_cut(int flow) {
  printf("%d\n", flow);

  for (int u = 1; u <= n; u++) {
    if ((u == 1) || (nd[u].parent != NIL)) {
      for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
        int v = e[pos].v;
        if ((nd[v].parent == NIL)) {
          printf("%d %d\n", u, v);
        }
      }
    }
  }
}

int main() {
  read_data();
  int max_flow = ford_fulkerson();
  write_min_cut(max_flow);

  return 0;
```

```
}
```

A.3 Problema Distinct Routes (CSES)

‹ înapoi

```
#include <queue>
#include <stdio.h>

const int MAX_NODES = 500;
const int MAX_EDGES = 1'000;
const int NIL = -1;
const int INFINITY = 1'001;

struct edge {
  short v;
  short cap;
  short next;
};

struct node {
  short adj;
  short parent, edge;
};

edge e[2 * MAX_EDGES];
node nd[MAX_NODES + 1];
std::queue<short> q;
int n, num_edges;

void add_edge(short u, short v, short cap) {
  static int pos = 0;

  e[pos] = { v, cap, nd[u].adj };
  nd[u].adj = pos++;
}

void read_data() {
  scanf("%d %d", &n, &num_edges);
  for (int u = 1; u <= n; u++) {
    nd[u].adj = NIL;
  }
  for (int i = 0; i < num_edges; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_edge(u, v, 1);
    add_edge(v, u, 0);
  }
```

```
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

bool bfs_reachable() {
  for (int u = 1; u <= n; u++) {
    nd[u].parent = NIL;
  }
  q = {};
  q.push(1);

  while (!q.empty() && (nd[n].parent == NIL)) {
    int u = q.front();
    q.pop();
    for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
      int v = e[pos].v;
      // Consideră doar capacitățile pozitive către noduri neexplorate.
      if ((nd[v].parent == NIL) && e[pos].cap) {
        nd[v].parent = u;
        nd[v].edge = pos;
        q.push(v);
      }
    }
  }

  return (nd[n].parent != NIL);
}

int path_minimum() {
  int min_cap = INFINITY;
  int u = n;
  while (u != 1) {
    min_cap = min(min_cap, e[nd[u].edge].cap);
    u = nd[u].parent;
  }
  return min_cap;
}

void pump_on_path(int flow) {
  int u = n;
  while (u != 1) {
    e[nd[u].edge].cap -= flow;
    e[nd[u].edge ^ 1].cap += flow;
    u = nd[u].parent;
  }
}

int ford_fulkerson() {
```

```
  int flow = 0;

  while (bfs_reachable()) {
    int augment = path_minimum();
    pump_on_path(augment);
    flow += augment;
  }

  return flow;
}

void clear_parent() {
  for (int u = 1; u <= n; u++) {
    nd[u].parent = NIL;
  }
}

void dfs(int u, int parent, int edge) {
  nd[u].parent = parent;
  nd[u].edge = edge;

  for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
    int v = e[pos].v;
    int rev = pos ^ 1;
    // Follow a regular edge if there is residual flow on the reverse edge.
    if ((nd[v].parent == NIL) &&
        !(pos & 1) &&
        e[rev].cap) {
      dfs(v, u, pos);
    }
  }
}

void print_path(int u, int len) {
  if (u == 1) {
    printf("%d\n1 ", len);
  } else {
    int edge = nd[u].edge;
    e[edge].cap++;
    e[edge ^ 1].cap--;
    print_path(nd[u].parent, len + 1);
    printf("%d ", u);
  }
}

void recover_paths() {
  do {
    clear_parent();
    dfs(1, NIL, NIL);
    if (nd[n].parent != NIL) {
```

```
      print_path(n, 1);
      printf("\n");
    }
  } while (nd[n].parent != NIL);
}

int main() {
  read_data();
  int max_flow = ford_fulkerson();
  printf("%d\n", max_flow);
  recover_paths();

  return 0;
}
```

A.4 Problema Soldier and Traveling (Codeforces)

❮ înapoi • versiune online

```
#include <stdio.h>

const int MAX_CITIES = 100;
const int MAX_ROADS = 200;
const int MAX_NODES = 2 * MAX_CITIES + 2;
const int MAX_EDGES = 3 * MAX_CITIES + 2 * MAX_ROADS;
const int INFINITY = 1'000;
const int NIL = -1;

struct cell {
  short v;
  short cap;
  short next;
};

struct node {
  short adj;
  short parent;
  short edge;

  bool is_reachable() {
    return (parent != NIL);
  }
};

struct queue {
  int v[MAX_NODES];
  int head, tail;
```

```
  void clear() {
    head = tail = 0;
  }

  void enqueue(int x) {
    v[tail++] = x;
  }

  int dequeue() {
    return v[head++];
  }

  bool is_empty() {
    return head == tail;
  }
};

cell e[2 * MAX_EDGES];
node nd[MAX_NODES + 1];
queue q;
int n, num_nodes, num_edges;
int source, sink;
int sum_source, sum_sink;

void add_edge(short u, short v, short c) {
  static int pos = 0;

  e[pos] = { v, c, nd[u].adj };
  nd[u].adj = pos++;
}

void add_edge_pair(short u, short v, short c) {
  add_edge(u, v, c);
  add_edge(v, u, 0);
}

void read_data() {
  int m, val;
  scanf("%d %d", &n, &m);
  num_nodes = sink = 2 * n + 2;
  source = 2 * n + 1;

  for (int u =  1; u <= num_nodes; u++) {
    nd[u].adj = NIL;
  }

  for (int u = 1; u <= n; u++) {
    scanf("%d", &val);
    add_edge_pair(source, u, val);
    sum_source += val;
```

```
  }

  for (int u = 1; u <= n; u++) {
    scanf("%d", &val);
    add_edge_pair(u + n, sink, val);
    sum_sink += val;
  }

  for (int u = 1; u <= n; u++) {
    add_edge_pair(u, u + n, INFINITY);
  }

  while (m--) {
    int u, v;
    scanf("%d %d", &u, &v);
    add_edge_pair(u, v + n, INFINITY);
    add_edge_pair(v, u + n, INFINITY);
  }
}

bool bfs_reachable() {
  for (int u = 1; u <= num_nodes; u++) {
    nd[u].parent = NIL;
  }
  nd[source].parent = source;
  q.clear();
  q.enqueue(source);

  while (!q.is_empty() && !nd[sink].is_reachable()) {
    int u = q.dequeue();
    for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
      int v = e[pos].v;
      if (!nd[v].is_reachable() && e[pos].cap) {
        nd[v].parent = u;
        nd[v].edge = pos;
        q.enqueue(v);
      }
    }
  }

  return nd[sink].is_reachable();
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

int path_minimum() {
  int min_cap = INFINITY;
  int u = num_nodes;
```

```
  while (u != source) {
    min_cap = min(min_cap, e[nd[u].edge].cap);
    u = nd[u].parent;
  }
  return min_cap;
}

void pump_on_path(int flow) {
  int u = num_nodes;
  while (u != source) {
    e[nd[u].edge].cap -= flow;
    e[nd[u].edge ^ 1].cap += flow;
    u = nd[u].parent;
  }
}

int edmonds_karp() {
  int flow = 0;

  while (bfs_reachable()) {
    int augment = path_minimum();
    pump_on_path(augment);
    flow += augment;
  }

  return flow;
}

void print_travel_matrix() {
  int dest[n + 1];
  for (int u = 1; u <= n; u++) {
    for (int v = 1; v <= n; v++) {
      dest[v] = 0;
    }
    for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
      int v = e[pos].v - n;
      int count = e[pos ^ 1].cap;
      dest[v] = count;
    }
    for (int v = 1; v <= n; v++) {
      printf("%d ", dest[v]);
    }
    printf("\n");
  }
}

int main() {
  read_data();

  if (sum_source != sum_sink) {
```

```
    printf("NO\n");
  } else {
    int max_flow = edmonds_karp();
    if (max_flow != sum_source) {
      printf("NO\n");
    } else {
      printf("YES\n");
      print_travel_matrix();
    }
  }

  return 0;
}
```

A.5 Problema Fox and Dinner (Codeforces)

❮ înapoi • versiune online

```
#include <stdio.h>

const int MAX_N = 200;
const int MAX_NODES = MAX_N + 2;
const int MAX_EDGES = (MAX_N / 2) * (MAX_N / 2) + MAX_N;
const int MAX_VAL = 10'000;
const int NIL = -1;

struct edge {
  short v;
  short cap;
  short next;
};

struct node {
  short adj;
  short parent, edge;
};

struct queue {
  int v[MAX_N + 2];
  int head, tail;

  void clear() {
    head = tail = 0;
  }

  void enqueue(int x) {
    v[tail++] = x;
  }
```

```
  int dequeue() {
    return v[head++];
  }

  bool is_empty() {
    return head == tail;
  }
};

short age[MAX_N + 1];
bool composite[2 * MAX_VAL + 1];
node nd[MAX_NODES];
edge e[2 * MAX_EDGES]; // include și muchiile reziduale
queue q;
bool used[MAX_N + 1];
unsigned char cycle[MAX_N][MAX_N];
int cycle_len[MAX_N];
int num_cycles;
int n, num_odd, source, sink;
bool possible;

void read_data() {
  scanf("%d", &n);
  for (int i = 1; i <= n; i++) {
    scanf("%hd", &age[i]);
    num_odd += age[i] % 2;
  }
  possible = (2 * num_odd == n);
}

void sieve() {
  for (int i = 2; i <= 2 * MAX_VAL; i++) {
    if (!composite[i]) {
      for (int j = 2 * i; j <= 2 * MAX_VAL; j += i) {
        composite[j] = true;
      }
    }
  }
}

void add_edge(short u, short v, short cap) {
  static int ptr = 0;

  e[ptr] = { v, cap, nd[u].adj };
  nd[u].adj = ptr++;
  e[ptr] = { u, 0, nd[v].adj };
  nd[v].adj = ptr++;
}
```

```
void build_graph() {
  source = 0;
  sink = n + 1;

  for (int u = 0; u <= n + 1; u++) {
    nd[u].adj = NIL;
  }

  for (int i = 1; i <= n; i++) {
    if (age[i] % 2) {
      add_edge(i, sink, 2);
    } else {
      add_edge(source, i, 2);
    }
  }

  for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
      if ((age[j] % 2) && !composite[age[i] + age[j]]) {
        add_edge(i, j, 1);
      }
    }
  }
}

int min(int x, int y) {
  return (x < y) ? x : y;
}

bool bfs_reachable() {
  for (int u = 0; u <= n + 1; u++) {
    nd[u].parent = NIL;
  }
  q.clear();
  q.enqueue(source);

  while (!q.is_empty() && (nd[sink].parent == NIL)) {
    int u = q.dequeue();
    for (int ptr = nd[u].adj; ptr != NIL; ptr = e[ptr].next) {
      int v = e[ptr].v;
      if ((nd[v].parent == NIL) && e[ptr].cap) {
        nd[v].parent = u;
        nd[v].edge = ptr;
        q.enqueue(v);
      }
    }
  }

  return (nd[sink].parent != NIL);
}
```

```
int path_minimum() {
  int u = sink;
  int min_cap = e[nd[sink].edge].cap;

  while (u != source) {
    min_cap = min(min_cap, e[nd[u].edge].cap);
    u = nd[u].parent;
  }
  return min_cap;
}

void pump_on_path(int flow) {
  int u = sink;
  while (u != source) {
    e[nd[u].edge].cap -= flow;
    e[nd[u].edge ^ 1].cap += flow;
    u = nd[u].parent;
  }
}

int edmonds_karp() {
  int flow = 0;

  while (bfs_reachable()) {
    int augment = path_minimum();
    pump_on_path(augment);
    flow += augment;
  }

  return flow;
}

int find_neighbor(int u) {
  used[u] = true;

  for (int ptr = nd[u].adj; ptr != NIL; ptr = e[ptr].next) {
    int v = e[ptr].v;
    if (!used[v] &&
        (((age[u] % 2 == 0) && !e[ptr].cap) ||
         ((age[u] % 2 == 1) && e[ptr].cap))) {
      return v;
    }
  }

  return NIL;
}

void collect_cycle(int u) {
  int len = 0;
```

```
  cycle[num_cycles][len++] = u;
  while ((u = find_neighbor(u)) != NIL) {
    cycle[num_cycles][len++] = u;
  }

  cycle_len[num_cycles++] = len;
}

void collect_cycles() {
  used[source] = used[sink] = true;

  for (int i = 1; i <= n; i++) {
    if ((age[i] % 2 == 0) && !used[i]) {
      collect_cycle(i);
    }
  }
}

void write_cycles() {
  printf("%d\n", num_cycles);
  for (int i = 0; i < num_cycles; i++) {
    printf("%d", cycle_len[i]);
    for (int j = 0; j < cycle_len[i]; j++) {
      printf(" %d", cycle[i][j]);
    }
    printf("\n");
  }
}

int main() {
  read_data();

  if (possible) {
    sieve();
    build_graph();
    possible = (edmonds_karp() == n);
  }

  if (possible) {
    collect_cycles();
    write_cycles();
  } else {
    printf("Impossible\n");
  }

  return 0;
}
```

A.6 Problema Asfalt (Lot 2011)

‹ înapoi • versiune online

```
#include <queue>
#include <stdio.h>

const int MAX_NODES = 500;
const int MAX_EDGES = 5'000;
const int INFINITY = 1'000'000'000;
const int DIR_SOURCE = 0;
const int DIR_SINK = 1;
const int NIL = -1;

struct cell {
  int c;
  short v;
  short next;
  bool forward;
};

struct node {
  short adj;
  short parent;
  short edge;
  int dist[2];

  bool is_reachable() {
    return (parent != NIL);
  }
};

struct heap_elt {
  int dist;
  short u;

  friend bool operator< (const heap_elt& a, const heap_elt& b) {
    return a.dist > b.dist;
  }
};

cell e[2 * MAX_EDGES];
node nd[MAX_NODES + 1];
std::priority_queue<heap_elt> h;
int n, num_edges, source, sink;

void add_edge(short u, short v, int c) {
  static int pos = 0;
```

```
  e[pos] = { c, v, nd[u].adj };
  nd[u].adj = pos++;
}

void read_data() {
  FILE *f = fopen("asfalt.in", "r");

  fscanf(f, "%d %d", &n, &num_edges);
  fscanf(f, "%d %d", &source, &sink);

  for (int u = 1; u <= n; u++) {
    nd[u].adj = NIL;
  }

  for (int i = 0; i < num_edges; i++) {
    int u, v, c;
    fscanf(f, "%d %d %d", &u, &v, &c);
    add_edge(u, v, c);
    add_edge(v, u, c);
  }

  fclose(f);
}

void relax_all(int u, int dir) {
  for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
    short v = e[pos].v;
    int new_dist = nd[u].dist[dir] + e[pos].c;
    if (new_dist < nd[v].dist[dir]) {
      nd[v].dist[dir] = new_dist;
      h.push({new_dist, v});
    }
  }
}

void dijkstra(short start, int dir) {
  for (int u = 1; u <= n; u++) {
    nd[u].dist[dir] = INFINITY;
  }

  nd[start].dist[dir] = 0;
  h.push({0, start});

  while (!h.empty()) {
    heap_elt top = h.top();
    h.pop();

    if (top.dist == nd[top.u].dist[dir]) {
      relax_all(top.u, dir);
    }
```

```
  }
}

bool is_min_path_edge(int u, int edge_cost, int v) {
  return nd[u].dist[DIR_SOURCE] + edge_cost + nd[v].dist[DIR_SINK] ==
    nd[source].dist[DIR_SINK];
}

void convert_to_flow_network() {
  for (int u = 1; u <= n; u++) {
    for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
      int v = e[pos].v, c = e[pos].c;
      // Let one node configure the capacities of both edges.
      // Write down which of the two edges goes forward.
      if (u < v) {
        e[pos].c = e[pos ^ 1].c = 0;
        if (is_min_path_edge(u, c, v)) {
          e[pos].c = 1;
          e[pos].forward = true;
        } else if (is_min_path_edge(v, c, u)) {
          e[pos ^ 1].c = 1;
          e[pos ^ 1].forward = true;
        }
      }
    }
  }
}

bool bfs_reachable() {
  std::queue<int> q;

  for (int u = 1; u <= n; u++) {
    nd[u].parent = NIL;
  }
  nd[source].parent = source;
  q.push(source);

  while (!q.empty() && (nd[sink].parent == NIL)) {
    int u = q.front();
    q.pop();

    for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
      int v = e[pos].v;
      if ((nd[v].parent == NIL) && e[pos].c) {
        nd[v].parent = u;
        nd[v].edge = pos;
        q.push(v);
      }
    }
  }
```

```
  return nd[sink].is_reachable();
}

void pump_on_path() {
  int u = sink;
  while (u != source) {
    e[nd[u].edge].c--;
    e[nd[u].edge ^ 1].c++;
    u = nd[u].parent;
  }
}

int edmonds_karp() {
  int flow = 0;

  while (bfs_reachable()) {
    pump_on_path();
    flow++;
  }

  return flow;
}

bool is_cross_cut_flow_edge(int u, int v, int pos) {
  return
    nd[u].is_reachable() &&
    !nd[v].is_reachable() &&
    e[pos].forward &&
    (e[pos ^ 1].c == 1);   // e[pos] is a saturated flow edge
}

void write_min_cut(int flow) {
  FILE* f = fopen("asfalt.out", "w");
  fprintf(f, "%d\n", flow);

  for (int u = 1; u <= n; u++) {
    for (int pos = nd[u].adj; pos != NIL; pos = e[pos].next) {
      int v = e[pos].v;
      if (is_cross_cut_flow_edge(u, v, pos)) {
        fprintf(f, "%d %d\n", u, v);
      }
    }
  }

  fclose(f);
}

int main() {
  read_data();
```

```
  dijkstra(source, DIR_SOURCE);
  dijkstra(sink, DIR_SINK);
  convert_to_flow_network();
  int max_flow = edmonds_karp();
  write_min_cut(max_flow);

  return 0;
}
```

A.7 Problema Echipe (Lot 2025)

❮ înapoi • versiune online

```
#include <algorithm>
#include <set>
#include <stdio.h>

typedef std::multiset<int> multiset;

const int MAX_N = 150'000;
const long long INF = 1'000'000'000'000'000ll;

// Structura grafului. A_ sînt clasele persoanelor, iar B_ sînt rolurile.
const int S = 0;
const int A_C = 1;
const int A_M = 2;
const int A_MC = 3;
const int A_I = 4;
const int A_IC = 5;
const int A_IM = 6;
const int A_IMC = 7;
const int B_C = 8;
const int B_M = 9;
const int B_I = 10;
const int B_IM = 11;
const int T = 12;
const int NIL = -1;
const int NUM_CLASSES = 8;
const int NUM_NODES = 13;
const int FIRST_A = 1;
const int LAST_A = 7;
const int FIRST_B = 8;
const int LAST_B = 11;

const int ADJ[NUM_NODES][NUM_NODES] = {
  /* S */     { A_C, A_M, A_MC, A_I, A_IC, A_IM, A_IMC, NIL },
  /* A_C */   { S, B_C, NIL },
  /* A_M */   { S, B_M, B_IM, NIL },
```

```
  /* A_MC */  { S, B_C, B_M, B_IM, NIL },
  /* A_I */   { S, B_I, B_IM, NIL },
  /* A_IC */  { S, B_C, B_I, B_IM, NIL },
  /* A_IM */  { S, B_M, B_I, B_IM, NIL },
  /* A_IMC */ { S, B_C, B_M, B_I, B_IM, NIL },
  /* B_C */   { A_C, A_MC, A_IC, A_IMC, T, NIL },
  /* B_M */   { A_M, A_MC, A_IM, A_IMC, T, NIL },
  /* B_I */   { A_I, A_IC, A_IM, A_IMC, T, NIL },
  /* B_IM */  { A_M, A_MC, A_I, A_IC, A_IM, A_IMC, T, NIL },
  /* T */     { B_C, B_M, B_I, B_IM, NIL },
};

struct person {
  int attr, risk;
};

// O rețea de flux. Ocupă circa 700 de octeți, deci o putem copia cu totul la
// nevoie.
struct network_flow {
  int cap[NUM_NODES][NUM_NODES];
  bool seen[NUM_NODES];
  int flow;

  void build_from_persons(person* p, int n) {
    for (int i = 0; i < n; i++) {
      cap[S][p[i].attr]++;
    }

    // capacități infinite de la A la B
    for (int u = FIRST_A; u <= LAST_A; u++) {
      for (int i = 0; ADJ[u][i] != NIL; i++) {
        int v = ADJ[u][i];
        if ((v >= FIRST_B) && (v <= LAST_B)) {
          cap[u][v] = n;
        }
      }
    }
  }

  void increase_sink_capacities() {
    for (int u = FIRST_B; u <= LAST_B; u++) {
      cap[u][T]++;
    }
  }

  void increase_source_capacity(int a) {
    cap[S][a]++;
  }

  void reset_dfs() {
```

```
  for (int u = 0; u < NUM_NODES; u++) {
    seen[u] = false;
  }
}

bool dfs(int u) {
  seen[u] = true;
  if (u == T) {
    return true;
  }

  for (int i = 0; ADJ[u][i] != NIL; i++) {
    int v = ADJ[u][i];
    if (cap[u][v] && !seen[v] && dfs(v)) {
      cap[u][v]--;
      cap[v][u]++;
      return true;
    }
  }

  return false;
}

void ford_fulkerson() {
  bool reachable;
  do {
    reset_dfs();
    reachable = dfs(S);
    flow += reachable;
  } while (reachable);
}

void remove_unit(int a_node) {
  if (cap[S][a_node]) {
    // Există persoane cu aceste atribute încă necuplate. Elimină pur și
    // simplu una dintre aceste persoane.
    cap[S][a_node]--;
  } else {
    // Toate persoanele cu aceste atribute sînt cuplate. Caută un b către
    // care se duce fluxul și elimină unitatea S->a->b->T.
    int b = FIRST_B;
    while (!cap[b][a_node]) {
      b++;
    }

    cap[a_node][S]--;
    cap[a_node][b]++;
    cap[b][a_node]--;
    cap[b][T]++;
    cap[T][b]--;
```

```
      flow--;
    }
  }
};

struct team_collection {
  multiset risks_in[NUM_CLASSES];
  multiset risks_out[NUM_CLASSES];
  long long total_in_risk; // menținut incremental

  void include(person p) {
    risks_in[p.attr].insert(p.risk);
    total_in_risk += p.risk;
  }

  void exclude(person p) {
    risks_out[p.attr].insert(p.risk);
  }

  // Înlocuiește un risc (care garantat există).
  void replace_value(int attr, int old_risk, int new_risk) {
    multiset& rin = risks_in[attr];
    multiset& rout = risks_out[attr];

    auto it = rin.find(old_risk);
    if (it != rin.end()) {
      rin.erase(it);
      rin.insert(new_risk);
      total_in_risk += new_risk - old_risk;
    } else {
      it = rout.find(old_risk);
      rout.erase(it);
      rout.insert(new_risk);
    }
  }

  // Asigură-te că nicio valoare din risks_in nu este mai mare decît nicio
  // valoare din risks_out.
  void resort_sets(int attr) {
    multiset& rin = risks_in[attr];
    multiset& rout = risks_out[attr];

    if (!rin.empty() && !rout.empty()) {
      int last_in = *rin.rbegin();
      int first_out = *rout.begin();
      if (last_in > first_out) {
        rin.erase(std::prev(rin.end()));
        rout.erase(rout.begin());
        rin.insert(first_out);
        rout.insert(last_in);
```

```
        total_in_risk += first_out - last_in;
      }
    }
  }

  // Mută un element de la excluse la incluse.
  void move_in(int attr) {
    auto it = risks_out[attr].begin();
    total_in_risk += *it;
    risks_in[attr].insert(*it);
    risks_out[attr].erase(it);
  }

  // Mută un element de la incluse la excluse.
  void move_out(int attr) {
    auto it = std::prev(risks_in[attr].end());
    total_in_risk -= *it;
    risks_out[attr].insert(*it);
    risks_in[attr].erase(it);
  }
};

network_flow g;
team_collection tc;
person p[MAX_N];
int ind[MAX_N]; // pentru sortarea după risc
int n, num_teams;

void read_people() {
  scanf("%d ", &n);
  for (int i = 0; i < n; i++) {
    p[i].attr = getchar() - '0';
    p[i].attr += 2 * (getchar() - '0');
    p[i].attr += 4 * (getchar() - '0');
    scanf("%d ", &p[i].risk);
  }
}

void find_num_teams() {
  network_flow orig_g;
  g.build_from_persons(p, n);
  do {
    orig_g = g;
    num_teams++;
    g.increase_sink_capacities();
    g.ford_fulkerson();
  } while (g.flow == 4 * num_teams);

  // Ultima creștere a modificat graful, apoi a eșuat.
  num_teams--;
```

```
  g = orig_g;
}

void sort_by_risk() {
  for (int i = 0; i < n; i++) {
    ind[i] = i;
  }
  std::sort(ind, ind + n, [](int a, int b) {
    return p[a].risk > p[b].risk;
  });
}

void find_min_cost_teams() {
  sort_by_risk();

  for (int i = 0; i < n; i++) {
    person& dude = p[ind[i]];
    network_flow orig_g = g;
    g.remove_unit(dude.attr);
    g.ford_fulkerson();
    if (g.flow == 4 * num_teams) {
      tc.exclude(dude);
    } else {
      tc.include(dude);
      g = orig_g;
    }
  }

  printf("%d %lld\n", num_teams, tc.total_in_risk);
}

// Încearcă să înlocuiască ultimul element inclus din del cu primul element
// exclus din ins. Operează pe graful gc, pe care îl inițializează cu o copie
// a lui g.
//
// Returnează noul risc, sau INF dacă înlocuirea este imposibilă sau neoptimă.
long long try_swap_pair(int del, int ins, network_flow* gc, long long best_so_far) {
  if (del == ins ||
      tc.risks_in[del].empty() ||
      tc.risks_out[ins].empty()) {
    return INF;
  }

  int del_risk = *tc.risks_in[del].rbegin();
  int ins_risk = *tc.risks_out[ins].begin();
  long long new_risk = tc.total_in_risk - del_risk + ins_risk;

  if (new_risk >= best_so_far) {
    return INF; // Este o schimbare în rău.
  }
```

```
  // Acum are sens să încercăm să refacem fluxul.
  *gc = g;
  gc->remove_unit(del);
  gc->increase_source_capacity(ins);
  gc->ford_fulkerson();

  return (gc->flow == 4 * num_teams) ? new_risk : INF;
}

// Încearcă pe rînd să schimbe fiecare pereche (del, ins) unde first_del ≤ del
// ≤ last_del şi first_ins ≤ ins ≤ last_ins. Unul dintre aceste intervale are
// lungime 0.
void try_swap(int first_del, int last_del, int first_ins, int last_ins) {
  int step_del = (last_del > first_del);
  int step_ins = (last_ins > first_ins);

  network_flow best_g = g, gc = g;
  long long best_risk = tc.total_in_risk;
  int best_del = 0, best_ins = 0;

  for (int del = first_del, ins = first_ins;
       (del <= last_del) && (ins <= last_ins);
       del += step_del, ins += step_ins) {
    long long new_risk = try_swap_pair(del, ins, &gc, best_risk);
    if (new_risk < best_risk) {
      best_risk = new_risk;
      best_g = gc;
      best_del = del;
      best_ins = ins;
    }
  }

  if (best_risk < tc.total_in_risk) {
    g = best_g;
    tc.move_out(best_del);
    tc.move_in(best_ins);
  }
}

void try_swap_out(int attr) {
  try_swap(attr, attr, FIRST_A, LAST_A);
}

void try_swap_in(int attr) {
  try_swap(FIRST_A, LAST_A, attr, attr);
}

void replace(int attr, int old_risk, int new_risk) {
  tc.replace_value(attr, old_risk, new_risk);
```

```
  tc.resort_sets(attr);
  if (new_risk > old_risk) {
    try_swap_out(attr);
  } else if (new_risk < old_risk) {
    try_swap_in(attr);
  }
}

void process_queries() {
  int num_queries, pid, risk;
  scanf("%d", &num_queries);
  while (num_queries--) {
    scanf("%d %d", &pid, &risk);
    pid--;
    replace(p[pid].attr, p[pid].risk, risk);
    p[pid].risk = risk;
    printf("%lld\n", tc.total_in_risk);
  }
}

int main() {
  read_people();
  find_num_teams();
  find_min_cost_teams();
  process_queries();

  return 0;
}
```